

The dirty model and the cohomology of cost: questions, options, recommendations

ACCEPTED (2026-08-10, Mathijs: “I agree with all points of the aip as suggested”) — all recommendations ratified: exchange-form $\mathbb{N}$ -cohomology (§1B), §2’s write-up, framework-first mechanization (§3), lazy-as-ground-truth with flush-inclusive interface statements (§5), the §9 sequencing. Amendment to §4 on acceptance: the classification runs *now* as a contained sol track (Mathijs’s call) — contained meaning: its only output is a report draft; it does not reorder the roadmap.

Rev 2 (2026-08-10, Claude; rev 1 was a bare list — this rev adds background and pros/cons per question, on Mathijs’s request). Pricing details remain a Mathijs call at cache-v0 call 2. Sources: Expo 3 (cocycle framing + three badge claims), cache-v0 §4b (dirty-up refinement; candidate in-machine resolution of [72]).

0. Background, in five sentences

The action category $\mathcal{C}$ has configurations as objects and pairs (w, c) as morphisms (Grothendieck construction of the word action); a cost model is a functor $\mathcal{C} \rightarrow B\mathbb{N}$, i.e. a function $\text{cost}_c(w)$ obeying the cocycle law. Word-only costs (homomorphisms) are the functors that factor through the projection $\pi : \mathcal{C} \rightarrow BM$; coboundaries $\delta\Phi$ are the potential functions of amortized analysis; two costs in the same cohomology class agree on every closed walk and differ only by boundary terms. The dirty model prices a write-back on leaving a written subtree: down fills, clean up discards free, dirty up pays. It reads one honest bit of local state, so it is not a homomorphism — and Expo 3 conjectures it is not even *cohomologous* to one: bandwidth cannot be statically priced. Everything below is the design space around that.

1. The $\mathbb{N}$ -coefficient question

The problem. Cohomology wants an abelian *group* of coefficients: a coboundary $(\delta\Phi)_c(w) = \Phi(c \cdot w) - \Phi(c)$ subtracts, and goes negative whenever the potential drops. Our costs are $\mathbb{N}$ -valued — the machine cannot pay -1 — so “ $\text{cost}' = \text{cost} + \delta\Phi$ ” is, read literally, an equation between an $\mathbb{N}$ -thing and a $\mathbb{Z}$ -thing. Expo 3 dodged this once by doubling ($2 \text{cost}_{\text{free}} = \text{cost}_{\text{sym}} + \delta\Phi$, everything $\mathbb{N}$). The question is what the honest general framework is, because Lean will force a choice.

Option A — work in $\mathbb{Z}$ (group completion). Embed $\mathbb{N}$ -cocycles in $\mathbb{Z}$ -cocycles, take ordinary $H^1(\mathcal{C}; \mathbb{Z})$, remember the positive cone on the side. *Pros*: textbook cohomology; every classical tool applies; “cohomologous” is a genuine equivalence in a genuine group. *Cons*: positivity is load-bearing for us, not a side-remark — lower bounds are order statements, and free-up’s pricing principle (“free is admissible exactly for potential-consuming operations”) is literally a cone condition. In $\mathbb{Z}$ the distinction between realizable cost models and formal differences evaporates; in Lean, Int-coercions infect every statement.

Option B — exchange form (recommended). Never write $\delta\Phi$ alone; state cohomology equations in the rearranged, subtraction-free form

$$\text{cost}'_c(w) + \Phi(c) = \text{cost}_c(w) + \Phi(c \cdot w).$$

This is exactly Tarjan’s amortized-analysis identity (amortized + potential-before = actual + potential-after): **N-valued cohomology in exchange form is not exotic — it is what amortized analysis has always been.** Any $\mathbb{Z}$ -relation between $\mathbb{N}$ -cocycles rearranges into such an $\mathbb{N}$ -equation (move negative terms across the equals sign; the doubling trick is one instance). *Pros:* fully $\mathbb{N}$ -native, mechanizes without coercions; the statements are the identities practitioners already know; positivity built in. *Cons:* H^1 is no longer a group — the classes form a preordered commutative monoid, and “cohomologous” must be checked as a two-sided relation. Some classical reflexes (subtract and compare to zero) need rephrasing.

Option C — per-grade boundedness (a dissolution, half-true). At fixed grade n the machine D_n is finite, so all potentials are bounded, and $\delta(\text{const} - \Phi) = -\delta\Phi$ makes negation available inside $\mathbb{N}$ (constants vanish under δ). So *per grade*, $\mathbb{N}$ vs $\mathbb{Z}$ is no real difference. *Pros:* reassuring — no per-grade pathology. *Cons:* the interesting statements are asymptotic *across* grades, and the trivializing constants blow up with n — which is exactly where the nontriviality conjecture lives (no potential *uniform in n* absorbs the gap). The dissolution is real but only per-grade; it sharpens rather than removes the question.

And one observation that defuses most of it (new, this rev). The class of a cocycle is determined by its values on *closed walks* — and closed-walk sums never subtract: the boundary term cancels. Comparisons of the form “these two round trips cost differently” are therefore $\mathbb{N}$ -native regardless of framework. Since the nontriviality conjecture is precisely a closed-walk statement (next section), the $\mathbb{N}$ -question matters for the general theory but *not* for the flagship theorem.

Recommendation: Option B for all mechanized statements; closed-walk (holonomy) comparisons for class-separation results; $\mathbb{Z}$ allowed in prose as ambient shorthand, never in Lean.

2. The nontriviality conjecture — and a near-proof

Background. Expo 3, open claim: no homomorphism h and potential Φ give $\text{cost}_{\text{dirty}} = h + \delta\Phi$. Witness shape: m isolated single-bit writes pay $\Theta(mn)$ in write-backs; the same write volume as one dense block pays $\Theta(2^k)$ — dirtiness coalesces.

The argument can be sharpened to nearly desk-proved (new, this rev). Two steps:

1. $B\mathbb{N}$ is *commutative*, so any homomorphism $h : M \rightarrow \mathbb{N}$ factors through the abelianization: h sees only the **letter counts** of a word, nothing about their order. (Letters are the raw generators — down0, down1, up, write, read; addresses are positions, not letters.)
2. Build the two scenarios as **closed walks with equal letter counts**: the sparse run (isolated writes, deep round trips) and the dense run padded with clean, write-free movement until the counts of every letter match. Both start and end at the root with a fully flushed tree, so *any* $\delta\Phi$ contributes zero; equal counts mean *any* h prices them equally; but their dirty costs differ by $\Theta(mn)$ vs $\Theta(2^k + \text{padding movement})$ — and the padding is clean movement whose dirty cost equals its letter cost, so the *gap* survives. Contradiction.

Executed 2026-08-10: the argument is written out in Expo 3 (with the erase-pass refinement that makes both walks genuine loops at one configuration c_0 , killing $\delta\Phi$ exactly — sparse writes 1s then restores 0s; dense sweeps and erases; both end root/clean/all-zero) and the claim is re-badged desk-proved there. The padding bookkeeping, exactly:

Take $m = 2^k$ with $m = \Theta(n)$. *Sparse walk:* $2m$ round trips (write pass + erase pass) to leaves in the left half chosen with balanced addresses ($n/2$ each of down0/down1 per path), none inside the block; each trip pays n downs + 1 write + n dirty ups, independent of the other trips. Totals: downs $2mn$ (split mn/mn), ups $2mn$, writes $2m$; dirty cost $4mn + 2m$. *Dense walk:* the leftmost 2^k -leaf block, entered by down0 $n-k$; one Euler sweep per pass writes/erases every leaf, crossing each block edge down once and up once ($2^k - 1$ left edges, $2^k - 1$ right). Core totals: downs $2[(n - k) + 2^{k+1} - 2]$, writes $2m$, write-backs $2[2^{k+1} - 2 + n - k] = \Theta(m + n)$. *Padding:* the letter deficits $\Delta_0, \Delta_1 > 0$ (and $\Delta_U = \Delta_0 + \Delta_1$ automatically, since both walks individually balance downs against ups) are spent as Δ_1 clean excursions into the never-written right half — each down1, then some down0s, then ups back to root, depth $\leq n$; capacity $\Delta_1(n - 1) \geq \Delta_0$ holds with room. Padding pays its downs but its ups are clean and free. Grand totals on equal letter counts: sparse $4mn + 2m$ versus dense-plus-padding $2mn + 2m + \Theta(m + n)$ — the gap $2mn - \Theta(m + n) = \Theta(n^2)$ is exactly the sparse walk’s dirty ups against the padding’s clean ones. No h (equal counts) and no Φ (equal endpoints) can see the difference. ◻

The claim is thus desk-proved in full; the mechanization target (AIP-5 §3) inherits a completely explicit witness. *Pro of doing it now:* the argument only uses the clean movement model plus the dirty up rule — it is robust to most call-2 pricing details (any pricing where isolated write-backs pay per-path and coalesced ones pay per-edge-once). *Con:* if call 2 changes *which* ups pay (e.g. commit-at- boundaries only), the padding argument needs re-checking — cheap to redo, but not free.

3. The badge worklist: what to mechanize when

Background. Expo 3 carries three badges: the free-up representative identity ($2 \text{cost}_{\text{free}} = \text{cost}_{\text{sym}} + \delta\Phi$, desk), the sparse–dense asymmetry (desk), the nontriviality conjecture (open; upgraded by §2).

Option — mechanize framework-first (recommended): the cocycle framework (cost-as-functor on action words, exchange form, homomorphism-iff-factors-through- π) plus the free-up identity are *pricing-independent* — they concern the clean model and are mechanizable today, one well-scoped track. *Pros:* immediate green badges on Expo 3’s first claim; the framework is shared infrastructure for every later cost variant (level prices, contention); zero call-2 risk. *Cons:* the framework alone proves nothing new about dirtiness; badge value only.

Option — wait for call 2, mechanize everything at once. *Pros:* one track, no rework risk. *Cons:* serializes on Mathijs; loses the cheap wins; the §2 argument shows most of the dirty content is robust to the open pricing details anyway.

Recommendation: framework + free-up identity as the next Lean track after slot 1; §2’s conjecture-to-desk write-up can precede call 2; sparse–dense constants and the gauge (§5) wait for call 2.

4. Classifying the local classes (“the cohomology of the machine”)

Background. The finitely-presentable cocycles are those given by a table: generator $\times$ *local observation* (dirtiness is one bit per node). Question: which finite local tables give nontrivial classes? Is write-back *the* generator of the local H^1 , or are there others (warmth counters, generation tags)?

Pros of chasing this: it would turn “cost is a cocycle” from a framing into an invariant — a computed H^1_{loc} is a statement about *the machine itself*, independent of any particular pricing; a clean answer (“local $H^1 = \text{homomorphisms} \oplus \langle \text{write-back} \rangle$ ”) would be genuinely beautiful and quotable; and it gives a *criterion* for future model extensions — a proposed resource is “genuinely new” exactly when its class is nonzero (contention, warmth, wear all become testable). *Cons:* real rabbit-hole risk — the action category is enormous, and the answer is relative to the observation language (change what “local” may read and the classification changes with it: garbage in, garbage out); there is no downstream consumer yet besides elegance; the classification could be hard in a way that produces no partial payoff. **Recommendation:** state precisely, do not chase yet. Trigger for activation: the moment a *second* concrete candidate class exists (e.g. a contention or warmth pricing from cache-v1), the pairwise question “same class or not?” becomes decision-relevant and the general machinery earns its keep.

5. The gauge: what do open runs owe?

Background. Cohomologous costs agree on closed walks and differ on open runs by the boundary term. Operationally: a run that ends with dirty subtrees has *unflushed debt*. The **lazy** gauge charges only work actually done; the **flush-inclusive** gauge adds $\delta(\text{dirty-count})$ — i.e. prices every run as if it committed at the end. Same cohomology class, different representative; this is cache-v0 §4b’s commit reading and the operational face of [72].

Lazy — pros: it is the machine’s raw operational cost; simplest mechanization; composes as-is (no per-seam correction). *Lazy — cons:* per-run statements can hide debt — a “cheap” run may owe an unbounded flush; cross-run comparisons (exactly the sparse–dense witness) need endpoint-cleanliness side-conditions. *Flush-inclusive — pros:* honest interfaces — a run’s price bounds its externalized cost; monotone under extension; matches durability semantics and makes [72]’s “writes twice” read as “pays its write-backs”, with zip’s read/write asymmetry falling out. *Flush-inclusive — cons:* charged at *every* statement boundary it would destroy coalescing (the whole point of dirtiness); it must be an interface-level convention, not a per-word one; slightly heavier statements (the correction term travels along).

Recommendation: mechanize the lazy cocycle as ground truth (it is the machine); *state* interface theorems in the flush-inclusive gauge by adding the explicit $\delta(\text{dirty-count})$ term. Because they are the same class, no machinery is duplicated — the gauge is a statement-level choice. Whether commit also becomes a machine *operation* (durability as semantics, not just pricing) is a call-2 question and stays open here.

6. Interaction with level prices $c(\ell) \neq 1$

Background. Under Euclidean pricing $c(\ell) = 2^{\ell/k}$, a write-back at level ℓ costs $c(\ell)$; the sparse–dense exponents shift (isolated writes pay $\Theta(N^{1/k})$ per item, coalesced regions amortize along

the boundary) but the asymmetry persists. The folklore anchors are exact: SSD/flash *write amplification* and the random-vs-sequential write gap are dirty coalescing observed in the wild, at $k \approx 2, 3$.

Pros of doing it early: the Euclidean variants are freshly decided anyway (cache-v0 §5), one track could carry both; the empirical berry can *measure* the write asymmetry cheaply (a seq-vs-random write microbench next to the read one). *Cons:* multiplies the surface while the $c \equiv 1$ dirty model is not yet fixed; the clean-model Euclidean variants (movement only) don't depend on dirtiness and shouldn't wait for it. **Recommendation:** sequence dirty- $\times$ -levels *after* the $c \equiv 1$ dirty mechanization; add the write microbench to the empirical berry now (cheap, independent).

7. Does the RAM correspondence survive? (mostly yes, cheaply)

Background + a small observation. Every write-back pays for an edge that some down filled, and each filled edge writes back at most once per fill; so total dirty surcharge $\leq$ total downs, i.e. $\text{cost}_{\text{dirty}} \leq 2 \cdot \text{cost}_{\text{clean}}$. Magnitudes are within a factor 2; the simulations ($\leq 5 \cdot T$ etc.) survive with adjusted constants. *Pro:* a cheap theorem, worth a receipt, and it sharpens the story: **dirtyness changes the cohomology class, not the magnitude** — the cocycle lens sees structure that Θ -bounds are blind to, which is exactly the paper's argument for carrying the lens at all. *Con:* not worth a standalone track; fold into the dirty mechanization as a lemma.

8. Write entropy is box-counting (new, this rev)

Background. The read side has the entropy theorem: frequent heads near, $\text{cost} \approx n(1 + H)$. What is the write-side twin? Writes don't care about *frequency*; they care about *spatial clustering* — coalescing is geometric.

The observation: the dirty cost of writing a set S of leaves is, up to constants, the number of edges of the dyadic closure of S — that is

$$\text{cost}_{\text{write}}(S) \approx \sum_{\ell} N_{\ell}(S),$$

where $N_{\ell}(S)$ counts the distinct level- ℓ ancestors of S . But N_{ℓ} is the *box count of S at scale $2^{-\ell}$* : the write cost of a set is its summed **box-counting profile**, and the growth exponent of N_{ℓ} is its fractal (Minkowski) dimension. Sparse (dim 0) vs dense (dim 1 in leaf space) recovers the sparse-dense asymmetry as the two endpoints of a dimension dial — and the fractal-dimension theme (shannon- programme §2: level prices as *ambient* dimension) reappears on the data side as the dimension of the *written set*. Candidate theorem: dirty cost of a write set is Θ of its box-counting sum; corollary: cost per written leaf is governed by the set's dimension. Interaction with the k -way cliff (phase 3): plausibly the cliff is a statement about the dimension profile of merged streams — pointer recorded, not developed. *Pros:* gives the dirty model its own quantitative theory, parallel to (not imitating) the read-side entropy theorem; highly measurable (write cost vs set dimension is a benchmarkable curve); ties the two fractal threads together. *Cons:* needs the dirty mechanization first; the box-counting sum is only meaningful once pricing is fixed (gauge of §5 shifts constants); risk of theory-before-model if pushed ahead of call

2. **Recommendation:** record now (this section), state after the

$c \equiv 1$ dirty track lands.

8b. Addendum (2026-08-10, post-acceptance): the calculus-tier boundary, pre-connected

The founding convo’s splay-tree exercise ([388]–[424]) recorded a boundary that belongs in this AIP’s orbit. It was *not* that potentials fail on splay trees (Sleator–Tarjan’s potential is fine); it is that the calculus’s filtration cost design cannot *prove* input-dependent amortized bounds: amortization-as-normalization resolves branches statically, which works only for oblivious access patterns; splay needs to charge across operation boundaries, and “ $\text{cost}(f;g) \leq \text{cost}(f) + \text{cost}(g)$ ” forbids exactly that transfer. The convo’s identified fix — **time credits as a linear resource** (Atkey; Charguéraud–Pottier’s union-find; Grodin–Harper in calf) — is, in this AIP’s language, precisely the **exchange form** (§1B): “amortized cost a given invariant Φ ” is $a + \Phi_{\text{before}} = \text{actual} + \Phi_{\text{after}}$, credits = the potential made affine. So the v2 credit judgment is the calculus-level face of the cocycle framework mechanized at the machine level (`lean/Adic/Cocycle.lean`); when the calculus arrives, its amortization layer should be *defined* as this exchange judgment rather than invented separately. Genuinely out of reach even then, per the convo: *relative* bounds (static/ dynamic optimality — cost of f vs cost of g on shared inputs needs a relational cost logic; parked). Note the machine level dodges this: universal mounting (Shannon move 2) quantifies over distance assignments, not program pairs, so its competitive statement stays first-order. Investigation deferred to the calculus tier; recorded here so the design lands pre-connected.

9. Suggested sequencing

1. §2’s padding argument written out $\rightarrow$ conjecture re-badged desk-proved (Claude desk work, small).
2. Cocycle framework + free-up identity in Lean (one track; pricing-independent).
3. Call 2 (Mathijs): dirty pricing details + gauge/commit semantics (§5) — unlocks sparse–dense mechanization and §8.
4. Dirty $\times$ level prices (§6) after both parents exist; write microbench joins the empirical berry immediately.
5. §4 (classification): dormant until a second candidate class exists.